

TP01_2025_Resolucion: Informe Técnico y Análisis de la Plataforma EDU-CIAA-NXP y el Microcontrolador LPC4337

El presente informe constituye la resolución exhaustiva de los puntos planteados en la Guía de Trabajos Prácticos Nº 1 (TP01_2025) del Taller de Proyecto I (E306). El análisis se centra en la plataforma de desarrollo EDU-CIAA-NXP, el microcontrolador NXP LPC4337, su arquitectura de *hardware* y la pila de *software* propuesta por la cátedra (CMSIS, LPCOpen, y sAPI).

I. Fundamentos de la Plataforma y Definición de Proyecto

1.1. La Plataforma EDU-CIAA: Características y Filosofía

La Plataforma CIAA (Computadora Industrial Abierta Argentina) se distingue conceptual y funcionalmente de otras placas de desarrollo de propósito general disponibles en el mercado, como Arduino o Raspberry Pi. Mientras que estas últimas están diseñadas principalmente para el prototipado, las demostraciones o el uso hobbysta, la CIAA fue concebida con la **robustez industrial** como prioridad fundamental.¹

Esta concepción industrial implica que el diseño de *hardware* de la CIAA debe soportar condiciones ambientales hostiles que son comunes en entornos de fábrica, incluyendo ruidos electromagnéticos, vibraciones, picos de tensión e interferencias.¹ La principal consecuencia de este enfoque es la inclusión de circuitos de protección y acondicionamiento de señal en las entradas y salidas, superando las limitaciones de fragilidad de las plataformas de desarrollo convencionales. Por ejemplo, los manuales de placas de *hobby* a menudo advierten contra la conexión de señales lógicas de 5 V o la aplicación de tensión a pines de entrada/salida cuando la placa no está alimentada, ya que esto puede dañar irreversiblemente el procesador.¹ El diseño de la CIAA busca mitigar precisamente estos riesgos.

La **EDU-CIAA-NXP** es una variante de bajo costo de la CIAA-NXP, específicamente optimizada para la enseñanza en niveles universitarios y terciarios.² Al basar la formación en una arquitectura con un microcontrolador de alto rendimiento (LPC4337) y una placa diseñada bajo estándares industriales, se logra un ambiente de aprendizaje que simula los desafíos reales y los requisitos de confiabilidad de la ingeniería profesional. Esta elección de plataforma obliga a los estudiantes a comprender y gestionar la complejidad inherente a los

sistemas empotrados de grado industrial.

Característica	CIAA (CIAA-NXP/EDU-CIAA)	Plataformas Hobby (ej. Arduino/Raspberry)
Concepción Principal	Producto robusto para ambiente industrial y educativo ¹	Prototipado, demostraciones, uso hobbysta ¹
Resistencia Ambiental	Alta (ruidos, vibraciones, picos de tensión, EMI) ¹	Baja, sensible a condiciones adversas ¹
Protección Eléctrica I/O	Diseñada para soportar condiciones hostiles	Riesgo de daño por voltajes fuera de rango o desconexión ¹

1.2. Periféricos de Expansión en Conectores P1 y P2

Para el desarrollo de aplicaciones en la EDU-CIAA-NXP, se utilizan conectores de expansión denominados P1 y P2.⁴ La determinación de los periféricos disponibles en estos conectores requiere un análisis directo del documento de especificaciones de la placa (EDU-CIAA-NXP v1.1 Board - 2019-01-03 v5r0.pdf) y del esquemático (EDU-CIAA-NXP Esquematico.pdf).²

En microcontroladores avanzados como el LPC4337, cada pin físico puede tener múltiples funciones alternativas (hasta ocho, de FUNC0 a FUNC7, como se detalla en el Capítulo 2.2). Por lo tanto, los conectores P1 y P2 exponen un amplio conjunto de estas funciones, que generalmente incluyen las siguientes categorías de periféricos:

1. **GPIOs de Propósito General:** Numerosos pines configurables como entrada o salida digital (la función GPIO es típicamente FUNC0).⁶
2. **Entradas y Salidas Analógicas:** Entradas para los Conversores Analógico-Digital (ADC) y posibles salidas del Conversor Digital-Analógico (DAC).⁶
3. **Interfaces de Comunicación Serie:** Líneas para protocolos estándar de baja velocidad (I2C, SPI, UART/USART) y de alta velocidad o industriales (CAN, Ethernet MII/RMII, SGPIO).⁶
4. **Funciones de Temporización:** Entradas de captura y salidas de comparación/PWM vinculadas a los temporizadores internos del LPC4337 (como el SCT).⁷

La correcta utilización de cualquier pin en P1 o P2 requiere que el desarrollador consulte el mapa de pines para identificar la conexión física (por ejemplo, P4_4) y configure la función deseada a través de la Unidad de Control del Sistema (SCU) antes de intentar acceder al periférico o al registro GPIO correspondiente.

1.3. Componentes Integrados: LEDs y Pulsadores

El análisis del circuito esquemático (EDU-CIAA-NXP Esquemático.pdf) ⁴ es necesario para comprender la interconexión de los componentes integrados clave: los LEDs (RGB, LED1, LED2, LED3) y los pulsadores (TEC1, TEC2, TEC3, TEC4). Estos componentes se utilizan como elementos básicos de entrada/salida para las primeras aplicaciones.

- **LEDs (Indicadores Luminosos):** Estos dispositivos están conectados a pines GPIO del LPC4337 configurados como salida. Es crucial determinar la lógica de conexión. En la mayoría de los diseños de sistemas empotrados, los LEDs se conectan en lógica negativa (*active low*), donde un estado lógico BAJO (0) en el pin GPIO enciende el LED, y un estado ALTO (1) lo apaga, permitiendo una gestión eficiente de la corriente. El LED RGB, en particular, requiere tres pines GPIO separados para controlar los canales Rojo, Verde y Azul.
- **Pulsadores (TECs - Teclas):** Los pulsadores de usuario están conectados a pines GPIO configurados como entrada. Para garantizar una lectura estable y definida, las entradas de los pulsadores requieren una polarización eléctrica que puede ser implementada de dos maneras:
 1. Resistencias *pull-up* o *pull-down* externas en la placa.
 2. Habilitación de resistencias *pull-up* o *pull-down* internas del microcontrolador a través del SCU.⁴

Si los pulsadores utilizan una resistencia *pull-up* (lo más común), el pin se lee como ALTO (HIGH) en reposo y como BAJO (LOW) cuando se presiona la tecla. El pinout documentado para la EDU-CIAA-NXP revela a qué banco y número de pin GPIO está mapeado cada TEC, información indispensable para acceder a su estado mediante el *software*.⁸

II. Análisis Arquitectónico Detallado del Microcontrolador NXP LPC4337

El LPC4337 es el cerebro de la plataforma EDU-CIAA-NXP y forma parte de la familia LPC43xx de NXP, caracterizada por su avanzada arquitectura multi-core y su rica colección de periféricos de alto rendimiento.

2.1. Arquitectura Interna del LPC4337 y Características Destacadas

El microcontrolador LPC4337 se distingue por su arquitectura **Dual-Core**, que integra dos núcleos de procesador ARM Cortex-M diferentes en el mismo chip ⁶:

1. **ARM Cortex-M4:** Es el núcleo principal de alto rendimiento. Este núcleo incluye una Unidad de Punto Flotante (FPU) y un conjunto de instrucciones de Procesamiento Digital de Señales (DSP), lo que lo hace ideal para la ejecución de algoritmos complejos, control de alto nivel y manejo del sistema operativo.⁴

2. **ARM Cortex-M0:** Es un núcleo secundario de baja latencia y alta eficiencia energética. Su función es típicamente gestionar tareas de control de periféricos en segundo plano, manejo rápido de interrupciones o protocolos de comunicación que requieren una respuesta determinista y rápida, liberando así al núcleo M4 para el procesamiento intensivo.⁶

La coexistencia de estos dos núcleos permite una distribución asíncrona de la carga de trabajo, una capacidad que mejora significativamente la robustez del sistema en tiempo real. Por ejemplo, el M0 puede dedicarse a la gestión de un bus CAN o de los puertos Serial GPIO (SGPIO) mientras el M4 ejecuta el control PID o algoritmos de interfaz gráfica. Esta sinergia multi-core es fundamental para desarrollar sistemas embebidos que demandan concurrencia y minimización de latencia.⁶

Además de los núcleos, el LPC4337 incorpora una amplia gama de periféricos de alta gama, incluyendo ⁶:

- **ADCHS:** Conversor Analógico-Digital de 12 bits.
- **SCT (State Configurable Timer):** Temporizador altamente flexible para control de motores y generación de PWM.
- **SPIFI (SPI Flash Interface):** Interfaz de memoria flash SPI.
- **C_CAN:** Módulos de Comunicación *Controller Area Network*.
- **Ethernet, USB, I2S, GPDMA** y un **RTC** (Reloj de Tiempo Real) con registros de respaldo alimentados por batería.⁷

2.2. Configuración de Terminales mediante el SCU (System Control Unit)

La versatilidad del LPC4337 se basa en la **Unidad de Control del Sistema (SCU)**, la cual es la encargada de la multiplexación de pines. Los pines físicos del microcontrolador son multipropósito, y el SCU determina qué función eléctrica y lógica se asigna a cada pin.

El SCU gestiona la multiplexación permitiendo seleccionar entre ocho funciones alternativas (FUNCO a FUNC7) para un mismo pin físico. Por ejemplo, la función **FUNCO** es típicamente asignada a la funcionalidad GPIO, mientras que otras funciones pueden corresponder a UART, SPI, Ethernet o control de memoria.⁶

El mecanismo de configuración de un pin es un proceso de dos etapas ineludible:

1. **Selección de la Función Principal (SCU):** Se debe escribir en el registro de configuración del SCU asociado al pin para seleccionar la función deseada (ej. seleccionar FUNCO para usarlo como GPIO).
2. **Configuración del Periférico/GPIO:** Una vez seleccionada la función, el periférico correspondiente (o el módulo GPIO) toma el control del pin y se configura a través de sus propios registros.⁶

Además de la multiplexación, el SCU controla las propiedades eléctricas de cada pin, incluyendo la activación de las resistencias de polarización internas (pull-up/pull-down) y el modo *flotante*. Es importante destacar que el SCU también interviene en la configuración de funciones analógicas; por ejemplo, para usar una entrada ADC, el pin debe configurarse primero como una entrada GPIO, y luego se utiliza un registro de selección de función analógica dentro del SCU para finalmente habilitar la capacidad del ADC.⁶

Si la configuración del SCU no se realiza correctamente en la primera etapa, el *software* fallará al intentar controlar el periférico asociado. Por esta razón, el *software* de inicialización del LPC4337 es inherentemente más complejo que el de microcontroladores sencillos, y requiere que la capa de abstracción de *hardware* (HAL, como sAPI) maneje esta secuencia de configuración con precisión.

2.3. Puertos GPIO: Funcionamiento a Nivel de Hardware

Los puertos GPIO (General Purpose Input/Output) del LPC4337 son la interfaz fundamental para interactuar con dispositivos digitales externos (LEDs, pulsadores, sensores).

Diagrama de Bloques Conceptual de un GPIO:

A nivel funcional, cada pin GPIO está conectado internamente a una estructura compleja que incluye:

1. **Bloque de Pin Multiplexing (SCU):** Controla qué señal interna se conecta al pin físico.
2. **Registro de Dirección (DIR):** Un bit de este registro determina si el pin opera como Entrada (0) o Salida (1).
3. **Etapas de Entrada:** Compuesta por un *buffer* y, en ocasiones, lógica de filtrado o histeresis para la lectura del estado físico del pin.
4. **Etapas de Salida:** Controlada por registros de datos (PIN, SET, CLR) que determinan el estado lógico de la señal que se presenta en el pin.

Una característica avanzada del LPC4337 es la inclusión de módulos de **Interrupción de Grupo GPIO**.⁷ Estos permiten generar una interrupción al núcleo cuando un patrón predefinido de estados lógicos es detectado en un grupo de pines. Esta funcionalidad es crucial para despertar al microcontrolador de modos de bajo consumo (aunque solo soporta *wake-up* desde el modo *Sleep*).⁶

2.4. Configuración y Control de GPIO como Salida

Para configurar y utilizar un pin GPIO como salida, se requiere:

1. **Configuración SCU:** Asegurar que el pin está configurado para la función GPIO (FUNC0) y que las resistencias de polarización internas están deshabilitadas o configuradas apropiadamente.
2. **Configuración de Dirección:** Establecer el bit correspondiente en el **Registro de**

Dirección (DIR) del puerto GPIO a 1.

La modificación del estado lógico de la salida puede realizarse de varias maneras, priorizando la eficiencia atómica del acceso al bus:

- **Escritura Directa (Registro PIN):** Se puede escribir el valor deseado (0 o 1) directamente en el Registro PIN. Sin embargo, esta operación requiere una lectura previa del estado completo del puerto para evitar modificar inadvertidamente otros pines que podrían estar configurados como entradas o salidas.
- **Registros SET y CLR:** La forma más eficiente y recomendada en arquitecturas ARM Cortex-M es utilizar los registros **SET (establecer)** y **CLR (limpiar)**. Escribir un '1' en el bit correspondiente del registro SET fuerza el pin a estado ALTO, mientras que escribir un '1' en el registro CLR fuerza el pin a estado BAJO. La ventaja es que estas son operaciones atómicas que solo afectan al bit deseado, sin necesidad de realizar una lectura-modificación-escritura (RMW) del puerto completo.

2.5. Configuración y Lectura de GPIO como Entrada

Para configurar y utilizar un pin GPIO como entrada, se requiere:

1. **Configuración SCU:** Configurar el pin como FUNC0 (GPIO) y, de manera crítica, definir el modo eléctrico de terminación: **Pull-up** (conectado internamente a VCC), **Pull-down** (conectado internamente a GND), o **Flotante**. La elección (pull-up/down) evita que la entrada permanezca en un estado indeterminado.
2. **Configuración de Dirección:** Establecer el bit correspondiente en el **Registro de Dirección (DIR)** del puerto GPIO a 0.
3. **Lectura del Estado:** El estado lógico del pin físico se lee del **Registro PIN**. Este valor es el resultado de la señal que ha pasado por el *buffer* de entrada del módulo GPIO y refleja el estado actual de la tensión externa.

2.6. SysTick Timer: Operación y Uso del Core ARM-Cortex M4

El **Systick Timer** es un temporizador simple, pero crucial, que está integrado directamente en el núcleo **ARM Cortex-M4**.⁹ Su estandarización en la arquitectura Cortex-M es lo que permite la portabilidad de los sistemas operativos y capas de *software* de tiempo real.

El SysTick opera como un **contador descendente de 24 bits**.⁹ Puede funcionar utilizando la frecuencia de reloj del procesador o una fuente de reloj de referencia interna.

Funcionamiento:

El temporizador se inicializa cargando un valor en el SysTick Reload Value Register (SYSTICK_RVR).⁹ Cuando la cuenta desciende del valor 1 al valor 0, se generan dos acciones 10:

1. Se genera una **Excepción SysTick** (interrupción) si esta está habilitada.
2. El contador se recarga automáticamente con el valor almacenado en SYSTICK_RVR,

iniciando el siguiente ciclo de conteo.⁹

La aplicación primordial del SysTick es generar una **interrupción periódica** que sirva de base de tiempo para un sistema operativo (OS) o para sistemas de *software* que implementan multitarea cooperativa. Esto permite al *kernel* ejecutar rutinariamente el cambio de contexto y la planificación de tareas.⁹

2.7. Control de Interrupciones NVIC

El **Nested Vectored Interrupt Controller (NVIC)** es el controlador de interrupciones del núcleo ARM Cortex-M4. Su función es gestionar todas las interrupciones externas (IRQs) y las excepciones internas generadas por el propio núcleo, priorizando, habilitando y vectorizando las peticiones.

El procesador Cortex-M4 opera en dos modos principales ¹⁰:

- **Thread mode:** Modo de procesamiento normal donde se ejecuta el código de aplicación principal.
- **Handler mode:** Modo activado para procesar excepciones e interrupciones.

Excepciones Propias del Core:

El NVIC gestiona un conjunto de excepciones que son intrínsecas a la arquitectura Cortex-M, incluyendo 10:

- Reset
- NMI (Non-Maskable Interrupt)
- Hard Fault, MemManage, BusFault, UsageFault (para gestión de errores del sistema)
- SVCall y PendSV (utilizadas por los sistemas operativos para llamadas al supervisor y cambio de contexto)
- La Excepción SysTick.

Operaciones de la CPU ante una Interrupción (Mecanismo de Cambio de Contexto):

Cuando la CPU recibe una petición de interrupción o excepción habilitada, realiza automáticamente las siguientes operaciones:

1. **Detección y Priorización:** El NVIC evalúa la prioridad de la interrupción recibida frente a la prioridad del código que se está ejecutando.
2. **Apilamiento Automático (Stacking):** Si la interrupción es aceptada, la CPU detiene la ejecución actual (Modo Thread) y guarda automáticamente el contexto mínimo de ejecución (registros R0-R3, R12, Link Register, Program Counter y Program Status Register) en la pila.¹⁰
3. **Vectorización:** El NVIC dirige la CPU a la dirección de memoria de la Rutina de Servicio de Interrupción (ISR) correspondiente, entrando en Modo Handler.
4. **Ejecución de la ISR:** Se ejecuta el código específico de la rutina (ej. SysTick_Handler(void) ¹⁰).
5. **Restauración Automática (Unstacking):** Al finalizar la ISR, la CPU restaura

automáticamente el contexto apilado, reanudando la ejecución del programa principal exactamente donde fue interrumpido. Este manejo automático del contexto por parte del hardware es un diferenciador clave de la arquitectura Cortex-M.

2.8. Periférico USART2 y Comunicación Serie

El microcontrolador LPC4337 incluye múltiples periféricos USART (Universal Synchronous/Asynchronous Receiver/Transmitter), de los cuales el **USART2** se utilizará en el Taller de Proyecto.

Características del USART2:

El USART2 es un periférico de comunicación serie versátil que permite la transmisión y recepción de datos en varios formatos (asíncrono/UART y síncrono). El *User Manual* de la familia LPC43xx indica que el USART2 posee funcionalidad avanzada:

- **Modo Síncrono:** Soporta la operación síncrona, utilizando pines específicos para el reloj (U2_UCLK).⁶
- **Soporte RS-485:** Incluye una señal de control de dirección (U2_DIR), lo que facilita la implementación directa del protocolo industrial RS-485/EIA-485.⁶

Un diagrama en bloques del USART típicamente incluye un generador de baudios (para control de velocidad), registros de desplazamiento para la serialización/deserialización (TSR/RSR), FIFOs de transmisión y recepción para almacenamiento temporal de datos, y la lógica de control que maneja la paridad, los bits de parada y la detección de errores.

Comparación con USART0 del MCU AVR ATMEGA328P:

El ATMEGA328P, utilizado en muchas plataformas de *hobby*, dispone únicamente de un **USART0**.¹² Si bien este periférico soporta comunicación asíncrona (UART) y síncrona básica, carece de varias de las características de alto rendimiento y las interfaces nativas del LPC4337. Específicamente, el USART0 del ATMEGA328P no ofrece soporte directo a nivel de pinout o registro para el control de dirección necesario para implementar RS-485 sin circuitería externa adicional. El LPC4337, al ofrecer múltiples USART y soporte nativo para RS-485 en pines como U2_DIR, demuestra su orientación a aplicaciones industriales y de redes complejas.

2.9. Conversores Analógico-Digital (ADC) Integrados en el LPC4337

El LPC4337 incorpora conversores Analógico-Digital de Alta Velocidad, referidos como **ADCHS** (Analog-to-Digital Converter High Speed).⁶

Características y Funcionamiento:

- **Resolución:** El ADCHS ofrece una resolución de **12 bits**.⁶ Esto significa que el rango de voltaje de entrada se divide en $2^{12} = 4096$ pasos de cuantificación.

- **Múltiples Entradas:** El conversor está configurado con múltiples canales de entrada analógica, conectadas a los bloques ADC0 y ADC1.⁶
- **Configuración SCU:** Como con otros periféricos, la señal de entrada analógica debe ser configurada a través del SCU, asegurando que el pin asociado no esté configurado como GPIO digital, sino que se active su función analógica.⁶

Comparación con ADC del ATMEGA328P:

La diferencia más significativa entre el LPC4337 y el ATMEGA328P radica en la precisión:

Característica	NXP LPC4337 (ADCHS)	ATMEGA328P (ADC)
Resolución	12 bits (4096 pasos) ⁶	10 bits (1024 pasos) ¹²
Canales	Múltiples (ADC0/ADC1) ⁶	6 canales ¹²
Tipo	Alta Velocidad (ADCHS)	Estándar

La resolución de 12 bits del LPC4337 proporciona una precisión de cuantificación cuatro veces superior a la del ADC de 10 bits del ATMEGA328P.⁶ Esta mayor resolución es crucial en aplicaciones de instrumentación y control industrial donde la adquisición de datos finos, con menor ruido de cuantificación, es indispensable para el monitoreo y control preciso de variables físicas.

III. Herramientas de Desarrollo y Gestión Temporal (Software Stack)

La programación de la EDU-CIAA-NXP se realiza utilizando una pila de *software* que implementa varias capas de abstracción para facilitar el desarrollo, desde el nivel del núcleo hasta la interfaz de la placa. Esta pila incluye CMSIS, LPCOpen, y sAPI.⁴

3.1. Estándares y Abstracciones de Software

A. CMSIS (Common Microcontroller Software Interface Standard)

CMSIS es un estándar desarrollado por ARM para proporcionar una capa de abstracción consistente e independiente del fabricante para los procesadores Cortex-M.¹⁴

- **Rol:** El objetivo principal es asegurar la portabilidad del código entre diferentes microcontroladores Cortex-M, reduciendo la curva de aprendizaje y el tiempo de

comercialización.¹⁴

- **CMSIS-CORE:** Proporciona interfaces y nombres de registros estandarizados para los componentes centrales del procesador, como el NVIC y el SysTick Timer. Por ejemplo, la función SysTick_Config(ticks) inicializa y pone en marcha el SysTick, configurándolo para generar interrupciones periódicas basadas en el número de *ticks* especificado, lo cual es la base para cualquier sistema operativo de tiempo real (RTOS).¹¹

B. LPCOpen

LPCOpen es la biblioteca de *software* de bajo nivel proporcionada por NXP (el fabricante) para sus familias de microcontroladores, incluyendo el LPC43xx.³

- **Rol:** Esta capa ofrece *drivers* directos para acceder a los periféricos específicos de NXP (GPIO, I2C, UART, SPI, ADC, etc.), así como *middleware* útil como pilas de red (LWIP) y librerías USB.¹⁵
- **Uso:** Los ejemplos de *blinky* incluidos en el proyecto Firmware_V3 demuestran cómo utilizar directamente las funciones de driver de LPCOpen para manipular el *hardware* LPC4337.⁴

C. sAPI (Simple API)

sAPI es una capa de abstracción de *hardware* (HAL) de alto nivel desarrollada específicamente para las placas CIAA (EDU-CIAA-NXP y CIAA-NXP), basada en LPCOpen.³

- **Rol:** El propósito de sAPI es simplificar aún más la programación, proporcionando funciones intuitivas que son específicas de los recursos disponibles en la placa (ej. gpioWrite(), tickRead()). Funciona como un intermediario entre el código de aplicación del usuario y los *drivers* específicos de NXP (LPCOpen), facilitando la gestión de tareas comunes de I/O y temporización.¹⁶

3.2. Gestión Temporal: Retardos Bloqueantes vs. No Bloqueantes

La gestión eficiente del tiempo es un desafío central en los sistemas embebidos, y el Taller de Proyecto distingue entre dos paradigmas fundamentales de temporización.

Retardo Bloqueante (Función delay()):

La función delay() implementa un retardo síncrono que generalmente se basa en un bucle de espera activa (*busy-waiting*) o en un temporizador configurado para generar una espera obligatoria.

- **Consecuencia:** Durante el retardo, el procesador dedica el 100% de sus ciclos a ejecutar el bucle de espera. Esto **bloquea** completamente el hilo de ejecución principal, impidiendo que la CPU responda a cualquier otro evento, interrupción o tarea de forma oportuna.⁴ En un sistema de tiempo real, los retardos bloqueantes son extremadamente perjudiciales, ya que pueden hacer que el sistema pierda datos o fallos al cumplir con sus

requisitos de tiempo estricto.

Retardo No Bloqueante (Funciones delayConfig(), delayRead()):

El enfoque no bloqueante utiliza el **Módulo Tick** de sAPI, que a su vez se basa en el SysTick Timer, para gestionar el tiempo de forma asíncrona.¹⁶

- **Mecanismo:** La función delayConfig() inicializa una estructura de tiempo estableciendo el inicio del conteo. La función delayRead() simplemente verifica si el tiempo deseado ha transcurrido consultando el valor actual del contador global de *ticks* (por ejemplo, milisegundos) gestionado por el SysTick.⁴
- **Consecuencia:** La CPU solo gasta ciclos de reloj mínimos en la llamada a delayRead(), permitiendo que el hilo principal continúe ejecutando otras operaciones (como leer sensores, gestionar comunicaciones o actualizar una máquina de estados finitos) entre la configuración y la verificación del retardo.¹⁷

La adopción de la temporización no bloqueante es una necesidad en sistemas avanzados como los basados en el LPC4337. Dado que el microcontrolador es dual-core y opera a alta velocidad, se espera que gestione múltiples tareas y responda rápidamente a eventos externos. Este requisito de concurrencia y respuesta inmediata se cumple mediante la implementación de *software* orientado a eventos o máquinas de estado, donde la temporización no bloqueante permite maximizar la utilización de la CPU.

Tabla: Análisis de Mecanismos de Retardo en sAPI

Función	Tipo de Retardo	Uso de CPU	Base de Tiempo	Adecuado para
delay()	Bloqueante	100% durante el retardo ⁴	Bucle de conteo o Timer	Inicialización simple, espera corta y no crítica
delayConfig() / delayRead()	No Bloqueante (Asíncrono) ⁴	Mínimo (solo lectura)	Módulo Tick (Systick) ¹⁶	Multitarea cooperativa, gestión de eventos, FSM

3.3. Control del Systick mediante sAPI (Tick Module)

El módulo **Tick** de sAPI actúa como la interfaz de usuario para el SysTick Timer del Cortex-M4.¹⁶ Proporciona un conjunto de funciones de alto nivel para configurar y manipular

el contador de tiempo del sistema de manera sencilla.

Las funciones clave son:

- `tickInit()`: Inicializa el SysTick Timer del core para generar una interrupción periódica a una frecuencia determinada (típicamente 1 ms), sentando la base para todos los servicios de tiempo del sistema.
- `tickRead()`: Permite leer el valor actual del contador de ticks, que es el valor base utilizado por las funciones de retardo no bloqueante como `delayRead()` para calcular el tiempo transcurrido.
- `tickWrite()`: Permite establecer un nuevo valor para el contador de ticks, lo que puede ser útil para resincronizar el reloj del sistema o para propósitos de prueba.
- `tickCallbackSet()`: Esta es una función esencial que permite al desarrollador asignar una rutina de usuario (*callback*) que se ejecutará cada vez que se dispare la interrupción del SysTick. Este mecanismo permite realizar tareas periódicas de fondo sin interferir con el bucle principal, logrando la concurrencia a nivel de *software* de aplicación.

El módulo Tick encapsula la complejidad del acceso a los registros CMSIS y NXP (SYSTICK_RVR, SYSTICK_CSR) de bajo nivel, lo que permite al desarrollador centrarse en la lógica de la aplicación.

Tabla: Arquitectura de Abstracciones de Software

Capa de Abstracción	Responsabilidad	Estandarización	Ejemplo de Función
Capa 3: sAPI	Abstracción de la placa (HAL), Programación orientada a eventos.	CIAA (Propia)	<code>tickRead()</code> , <code>gpioWrite()</code> ¹⁶
Capa 2: LPCOpen	Drivers de periféricos específicos de NXP (LPC43xx), Middleware.	Fabricante (NXP)	Funciones de driver para UART, ADC, etc. ¹⁵
Capa 1: CMSIS-CORE	Abstracción del Core ARM Cortex-M4/M0.	Estándar ARM	<code>SysTick_Config()</code> ¹¹

Capa 0: Hardware	SCU, GPIO Registers, Periféricos (ADCHS, USART2).	NXP LPC4337	Acceso directo a direcciones de memoria.
-------------------------	---	-------------	--

IV. Conclusiones y Propuesta para la Fase de Diseño

El análisis exhaustivo del TP01_2025 confirma que la plataforma EDU-CIAA-NXP y el microcontrolador LPC4337 representan un entorno de desarrollo de sistemas embebidos avanzado, diseñado para el cumplimiento de requisitos de tiempo real y la simulación de ambientes industriales.

El diseño del LPC4337 introduce una complejidad a nivel de *hardware* que requiere una comprensión profunda de las interdependencias arquitectónicas. La utilización de la arquitectura **dual-core (M4/M0)** permite la especialización en la gestión de tareas, donde el núcleo M0 puede descargar la responsabilidad de eventos rápidos de baja latencia del M4, mejorando la determinismo global del sistema.⁶

Críticamente, la configuración de cualquier funcionalidad en el LPC4337 está supeditada al **System Control Unit (SCU)**, que actúa como el árbitro de la multiplexación de pines.⁶ Un error en la configuración del SCU (etapa 1) hace que la configuración posterior del periférico o GPIO (etapa 2) sea inoperante. Este procedimiento de dos pasos es el principal punto de diferenciación de la inicialización de este MCU respecto a plataformas más sencillas.

En términos de capacidad, el LPC4337 supera al microcontrolador de referencia (ATMEGA328P) con una resolución de ADC de 12 bits y soporte nativo para protocolos industriales como RS-485 a través del USART2.⁶ Estas características subrayan la orientación del proyecto hacia aplicaciones de instrumentación y comunicación robustas.

Finalmente, la estructura de *software* (CMSIS $\rightarrow$ LPCOpen $\rightarrow$ sAPI) proporciona las herramientas necesarias para gestionar esta complejidad. La decisión de utilizar las funciones de **temporización no bloqueante** (delayRead()) basadas en el SysTick Timer es esencial para el éxito en la fase de diseño.⁴ La programación debe adoptar un paradigma orientado a eventos y máquinas de estado para garantizar que el sistema mantenga la capacidad de respuesta y la concurrencia necesarias para un sistema embebido de alto rendimiento.

Obras citadas

1. industria [] - Proyecto CIAA, fecha de acceso: diciembre 10, 2025, <https://www.proyecto-ciaa.com.ar/devwiki/doku.php%3Fid=industria.html>
2. EDU-CIAA-NXP: Plataforma Educativa ARM | PDF | Placa de circuito ..., fecha de

- acceso: diciembre 10, 2025,
<https://es.scribd.com/document/395106746/Edu-Ciaa-Nxp>
3. Releases · epernia/firmware_v3 - GitHub, fecha de acceso: diciembre 10, 2025,
https://github.com/epernia/firmware_v3/releases
 4. TP01_2025.pdf
 5. Edu ciaa-nxp pinout-a4_v4r3_es | PDF - Slideshare, fecha de acceso: diciembre 10, 2025,
<https://www.slideshare.net/slideshow/edu-ciaanxp-pinouta4v4r3es/109748861>
 6. UM10503 LPC43xx ARM Cortex-M4/M0 multi ... - NXP Community, fecha de acceso: diciembre 10, 2025,
<https://community.nxp.com/pwmxy87654/attachments/pwmxy87654/lpc/22545/1/UM10503.pdf>
 7. LPC4337FET256|Arm Cortex-M4/M0|32-bit MCU | NXP Semiconductors, fecha de acceso: diciembre 10, 2025, <https://www.nxp.com/products/LPC4337FET256>
 8. Manual Tarjeta CPLD | PDF | Corriente eléctrica | Point and Click - Scribd, fecha de acceso: diciembre 10, 2025,
<https://es.scribd.com/document/67744114/Manual-Tarjeta-Cpld>
 9. Tick Tick – ARM Cortex-M SysTick timer - IoTality, fecha de acceso: diciembre 10, 2025, <https://www.iotality.com/armcm-systick-timer/>
 10. ARM Cortex-M4 User Guide (Interrupts, exceptions, NVIC) STM32L4xx Microcontrollers Technical Reference Manual, fecha de acceso: diciembre 10, 2025,
https://www.eng.auburn.edu/~nelson/courses/elec5260_6260/slides/ARM%20STM32F476%20Interrupts.pdf
 11. CMSIS-Core (Cortex-M): SysTick Timer (SYSTICK) - GitHub Pages, fecha de acceso: diciembre 10, 2025,
https://arm-software.github.io/CMSIS_6/main/Core/group_SysTick_gr.html
 12. ATmega328P Microcontroller Pinout, Features & Datasheet - Vyrian, fecha de acceso: diciembre 10, 2025,
<https://www.vyrian.com/blog/atmega328p-microcontroller-features-pinout-configuration-datasheets/>
 13. ATMEGA328P Datasheet Overview: A Comprehensive Guide - AIChipLink, fecha de acceso: diciembre 10, 2025,
https://aichiplink.com/blog/ATMEGA328P-Datasheet-Overview-A-Comprehensive-Guide_117
 14. Common Microcontroller Software Interface Standard (CMSIS) - Arm, fecha de acceso: diciembre 10, 2025, <https://www.arm.com/technologies/cmsis>
 15. LPCOpen Libraries and Examples - NXP Semiconductors, fecha de acceso: diciembre 10, 2025,
<https://www.nxp.com/design/design-center/software/development-software/mcuxpresso-software-and-tools-/lpcopen-libraries-and-examples:LPC-OPEN-LIBRARIES>
 16. epernia/sAPI - GitHub, fecha de acceso: diciembre 10, 2025,
<https://github.com/epernia/sAPI>
 17. Treboada/AsyncBlinker: Non-blocking C/C++ library to manage blinking devices

- (for example, a LED) changing the state in ON-OFF time intervals. - GitHub, fecha de acceso: diciembre 10, 2025, <https://github.com/Treboada/AsyncBlinker>
18. Naveltay/NonBlockingDelay: Provides non-blocking delay functionality, allowing for timed operations without halting program execution in embedded systems projects. - GitHub, fecha de acceso: diciembre 10, 2025, <https://github.com/Naveltay/NonBlockingDelay>